

Three-Player Prisoner's Dilemma

Wonmin Song, Benjamin Parry, Justin Sculley, Zachary Bethancourt

Florida State University

MAP4103

Dr. Cogan

December 15, 2023

Abstract

The Three-Player Prisoner's Dilemma is an extension of the classic two-person prisoner's dilemma, a concept often used in game theory to analyze strategic interactions. In this scenario, three individuals are faced with a choice that involves cooperation and betrayal, leading to different outcomes based on their decisions. This paper analyzes the three-person prisoner's dilemma using iterated games, where an initial strategy is chosen and the effective "win rates" of each player are calculated after a certain number of games. The strategies investigated involve the player either defecting (turning away from their opponents and offering to testify) or cooperating (trusting their opponents and remaining silent). Of the 7 strategies in this paper, one emerges as the dominant force for a low number of iterations, and a few viable options emerge for a much higher number of games.

The winner for the highest low-iteration score is the "allDefect" strategy, in which the player always defects. However, as the number of games goes from 1 to 10 to 50, the allDefect strategy's effectiveness halves. The dominating strategies for higher numbers of games (up to 500 in this model) were "allCooperate" and "goByMajority". The allCooperate strategy means the player always trusts his fellow prisoners, gaining their trust after many iterations of the game. The goByMajority strategy keeps track of how many times their opponents have defected and cooperated, and decides to cooperate if the number of cooperations exceeds the number of defections (and vice versa).

These strategies are mathematically the best after many iterations, but some other complex strategies remained nearly constant in effectiveness (with a high score) as the number of iterations increased. For example, the win-score of the "softTfT" and "hardTfT" strategies varied by only a few points from 1 iteration to 500.

The results of this model imply that for a low number of iterations, corresponding to a lack of knowledge of your opponents in the real world, it is best for a player to defect. In the real world, if you have been in the prisoner's dilemma situation before, using that knowledge it may be better to use the goByMajority strategy. This model provides an interesting look into the logical methods a player can use in the prisoner's dilemma, and is a testament to the subtleties of human social competition.

Introduction

In the realm of game theory, the classic paradigm of the prisoner's dilemma has predominantly revolved around the interactions between two players. However, the strategic landscape becomes more intricate when expanding the scenario to involve three individuals. In this paper, we explore the complexities that might arise when three participants navigate the delicate balance between cooperation and betrayal.

Rule

In analyzing the payoffs for the three-player game, three key rules are established. First, defection is mandated as the dominant choice for each player, ensuring that it is always advantageous for a player to defect, regardless of the actions of their opponents. This principle is manifested in three constraints: $DCC > CCC$ (both opponents cooperate), $DDD > CDD$ (both opponents defect), and $DCD > CCD$ (one opponent cooperates, one defects). Second, a player should consistently benefit more if a greater number of opponents choose to cooperate. This guideline is reflected in the inequalities: $DCC > DCD > DDD$ and $CCC > CCD > CDD$. Third, when one player's choice is fixed, the other two players are subjected to a two-player prisoner's dilemma, outlined through the constraints $CCD > DDD$, $CCC > DCD$, $CCD > \frac{(CDD+DCD)}{2}$, and $CCC > \frac{(CCD+DCC)}{2}$. These rules provide a structured framework for understanding the strategic dynamics and constraints inherent in the three-player prisoner's dilemma, with specific numerical values assigned to the corresponding payoff outcomes (O'Donnell, 1998).

Mathematics

Using a notation system with variables D (Defection) and C (Cooperation) and the rules, we can construct a matrix that captures the outcomes resulting from the independent choices of each player.

	CCC	CCD	CDC	CDD	DCC	DCD	DDD
CCC	7	3	3	9	7	5	1
CCD	7	3	3	9	7	5	1
CDC	7	3	3	9	7	5	1
CDD	7	3	3	9	7	5	1
DCC	9	3	5	1	9	5	1
DCD	9	3	5	1	9	5	1
DDD	9	3	5	1	9	5	1

This matrix outlines the payoffs for various combinations, ranging from mutual cooperation to simultaneous betrayal. Each cell in the matrix corresponds to the payoff for a specific combination of choices made by the three players, where the rows represent the choices of player A, the columns represent the choices of player B, and the depth represents the choices of player C. The numerical values in the matrix are determined by the rules and constraints outlined for the three-player game.

Model Code

Score Matrix:

Cypher Key:

((c c c) (7 7 7))	////////////////////////////////////
((c c d) (3 3 9))	// 1 = allDefect //
((c d c) (3 9 3))	// 2 = allCooperate //
((d c c) (9 3 3))	// 3 = random //
((c d d) (0 5 5))	// 4 = goByMajority //
((d c d) (5 0 5))	// 5 = softTfT //
((d d c) (5 5 0))	// 6 = toughTfT //
((d d d) (1 1 1)))	// 7 = adaptiveTfT/ATFT //
	////////////////////////////////////

The score matrix and cypher key are used in the addScore() and cypher()/decypher() functions, respectively.

The score matrix is used to assign the appropriate payout to each player given the eight possible outcomes. It is read as follows:

First column = player one
 Second column = player two
 Third column = player three

An example usage is:

A round with choices (c, c, d) means players 1 and 2 chose c, and player 3 chose d, awarding players 1 and 2 three-points and player 3 nine-points.

The cypher key is implemented to allow the convenience of using integer representations for the different strategies. The function cypher() takes an integer as the parameter and returns the appropriate string name. Then, decypher() takes a string as the parameter and returns the appropriate integer.

So, for example:

cypher(1) returns “allDefect” and decypher(“allDefect”) returns 1

Partial 3 Person Class Declaration:

The following class declaration defines the functions needed for the 3 Person Prisoner's Dilemma game. Most notably, a struct data type is created for each player, containing their choice history, stored as a vector of ints, number of times they have chosen to defect and cooperate, their score, and their strategy. You can find the formal declarations for addScore(), cypher() and decypher() listed below, but we will not be looking at their definitions. However, the declarations for each strategy, and callStrat() are declared below, and will get their own sections.

```
class PD{

private:

    struct Player{

        std::vector<int> hist;    //previous choices
        int d;                  //number of defects
        int c;                  //number of cooperations
        int score;              //individuals score
        std::string strat;      //current employed strategy
    };

    Player P1, P2, P3;

public:

    //constructors

    PD();
    PD(std::string s1, std::string s2, std::string s3);
    PD(int s1, int s2, int s3);

    //strategies

    int allDefect() const;
    int allCooperate() const;
    int random() const;
    int goByMajority(int round, const Player &P1, const Player & P2, const Player & P3 ) const;
    int toughTfT(int round, const Player &P1, const Player & P2) const ;
```

```

int softTfT(int round, const Player &P1, const Player & P2) const;
int ATfT(int round, const Player &P1, const Player & P2 , double r) const;

//modifiers

void addScore(int c1, int c2, int c3);           //calculates each player's score after a round
void callStrat(int strat, Player &m, Player &o1, Player &o2, int r); //returns the appropriate
choice
void emptyHist();           //clears hist and sets d,c, and choice to zero

//helper functions

std::string cypher(int i) const;           //converts integer 1 through 7 to appropriate string name
int decypher(std::string strat) const;           //converts strategy string name to appropriate int

//play game: player 1-3 strategies and number of iterated games

void play(std::string strat1, std::string strat2, std::string strat3, int rounds);

};

```

Function Definitions for Strategies:

While we will look at the function definitions for each strategy, we should first look at a logical description of each function:

Function Name:	Description:
allDefect	Defects on every move.
allCooperate	Cooperates on every move.
Random	Makes a random move.

goByMajority	Cooperates on the first round, then on all other rounds, examines the history of the other player's actions, counting the total number of defections and cooperations. If the other player's defections outnumber their cooperations, it will defect; otherwise cooperate.
softTfT	Cooperates on the first round, then defects if <i>both</i> of the other players defected in the last round. Otherwise cooperate.
toughTfT	Cooperates on the first round, then defects if <i>either</i> of the other players defected in the last round. Otherwise cooperate.
adaptiveTfT	An adaption rate r is used to compute a continuous variable 'world' according to the history moves of the opponent.

```
// 0 = cooperate, 1 = defect
int PD::allDefect ( ) const { return 1; }
int PD::allCooperate ( ) const { return 0; }
int PD::random ( ) const { return random_gen(0,1); }
int PD::goByMajority ( int round, const Player &m, const Player & o1, const Player & o2 ) const
{
    if ( round <=1 )
        return 0;

    if (o1.d > m.c || o2.d > m.c)
        return 1;
    else
        return 0;
}

int PD::toughTfT ( int round, const Player &o1, const Player & o2 ) const
```

```

{
    if ( round<=1 )
        return 0;
    if (o1.hist.back() == 1 || o2.hist.back() == 1 )
        return 1;
    else
        return 0;
}
int PD::softTfT ( int round, const Player &o1, const Player & o2 ) const
{
    if ( round<=1 )
        return 0;
    if (o1.hist.back() == 1 && o2.hist.back() == 1 )
        return 1;
    else
        return 0;
}
int PD::ATFT ( int round, const Player &o1, const Player & o2, double r ) const
{
    double world = 0;
    if ( round<=1 )
        return 0;
    if ( o1.hist.back() == 0 || o1.hist.back() == 0 )
        world = world + r*(1-world);
    else
        world = world + r*(0-world);

    if ( world >= 0.5 )
        return 0;
    else
        return 1;
}

```

callStrat Member Function:

callStrat uses a switch statement to select the correct choice given player m 's strategy, and allows for the reuse of the same code instead of having repeated code for each player. The function takes the strategy *strat* (as an integer), the three players (with m being the main player), and the current round r . Given *strat* the correct case runs and calls the respective strategy

function, which then determines if the player's strategy means cooperate or defect for the given round, inserts that choice into the vector and updates their cooperation or defection count.

```
void PD::callStrat ( int strat, Player & m, Player & o1, Player & o2, int r )
{
    int choice=0;
    switch ( strat )
    {
        case 1:
            m.hist.push_back ( allDefect( ) );
            m.d++;
            break;
        case 2:
            m.hist.push_back ( allCooperate( ) );
            m.c++;
            break;
        case 3:
            choice = random();
            m.hist.push_back ( choice );
            if ( choice==1 )
                m.d++;
            else
                m.c++;
            break;
        case 4:
            choice = goByMajority(r,m,o1,o2);
            m.hist.push_back ( choice );
            if ( choice==1 )
                m.d++;
            else
                m.c++;
            break;
        case 5:
            choice = toughTfT ( r, o1, o2 );
            m.hist.push_back ( choice );
            if ( choice==1 )
                m.d++;
            else
                m.c++;
    }
}
```

```

        break;
    case 6:
        choice = softTfT ( r, o1, o2 );
        m.hist.push_back ( choice );
        if ( choice==1 )
            m.d++;
        else
            m.c++;
        break;
    case 7:
        choice = ATFT ( r, o1, o2, 0.1);
        m.hist.push_back ( choice );
        if ( choice == 1)
            m.d++;
        else
            m.c++;
        break;
    default:
        std::cout<<"error";
};
}

```

Lastly, the play function combines the functions above to play for *rounds* iterated games. It takes three strategies and the number of rounds as parameters and makes calls to callStrat for each player and updates the scores; repeats this for *rounds* times.

```

void PD::play ( std::string strat1, std::string strat2, std::string strat3, int rounds )
{
    P1.strat = strat1;
    P2.strat = strat2;
    P3.strat = strat3;

    for ( int i=1; i<=rounds; i++ )
    {
        callStrat ( decypher ( P1.strat ), P1, P2, P3, i );
        callStrat ( decypher ( P2.strat ), P2, P1, P3, i );
        callStrat ( decypher ( P3.strat ), P3, P1, P2, i );
        addScore ( P1.hist.back(), P2.hist.back(), P3.hist.back() );
    }
}

```

To evaluate the effectiveness of each strategy, we implemented the Prisoner's Dilemma game in 3 nested for loops to test each strategy against all possible combinations of games. We did this for a different number of iterated games to see how the strategies did over more consecutive games. For each number of rounds or iterated games, the average was found for each of the seven strategies by adding up player 1's scores in each game and dividing that total by the number of rounds and the number of trials, which in this case was 100.

Here is an example of the first 5 combinations and the last with rounds=5:

Game 1: (allDefect, allDefect, allDefect)
 Player 1: 5
 Player 2: 5
 Player 3: 5
 Game 2: (allDefect, allDefect, allCooperate)
 Player 1: 25
 Player 2: 25
 Player 3: 0
 Game 3: (allDefect, allDefect, random)
 Player 1: 17
 Player 2: 17
 Player 3: 2
 Game 4: (allDefect, allDefect, goByMajority)
 Player 1: 9
 Player 2: 9
 Player 3: 4
 Game 5: (allDefect, allDefect, softTfT)
 Player 1: 9
 Player 2: 9
 Player 3: 4
 ...
 ...
 ...
 ...
 ...

Game 343: (ATFT, ATFT, ATFT)

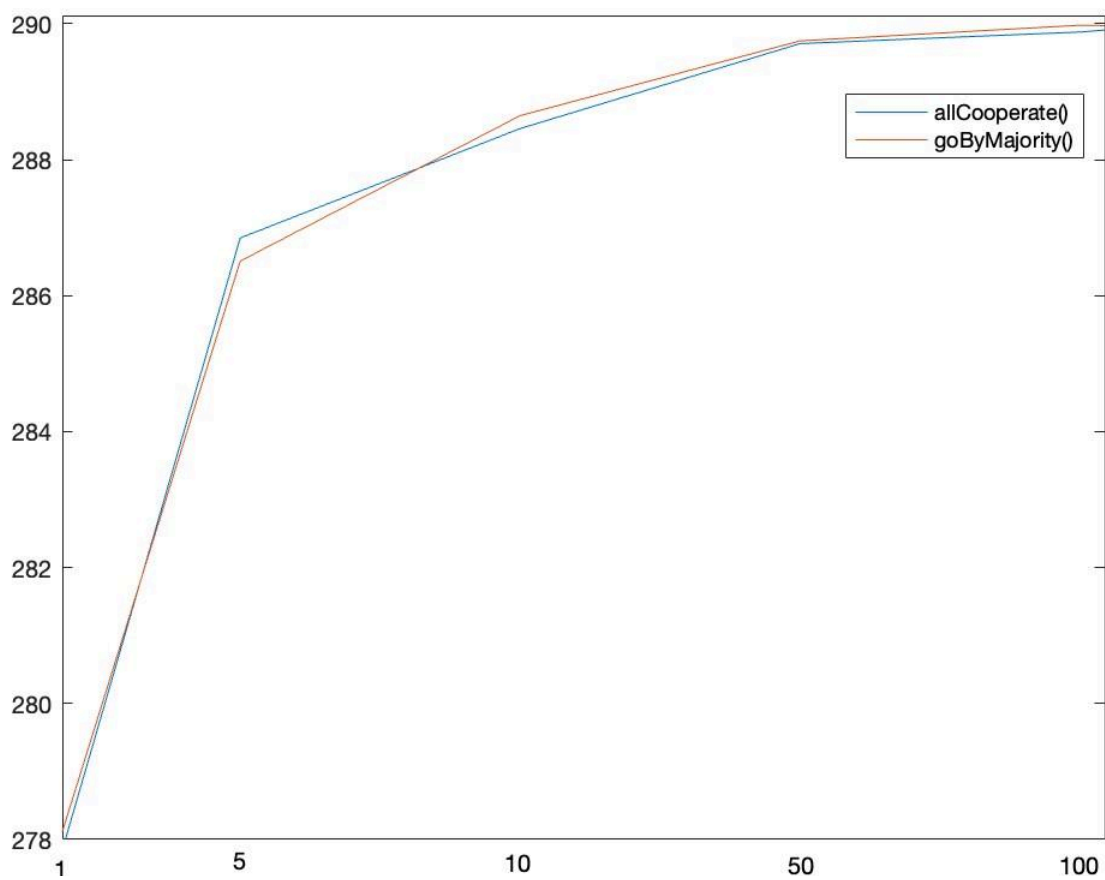
Player 1: 11

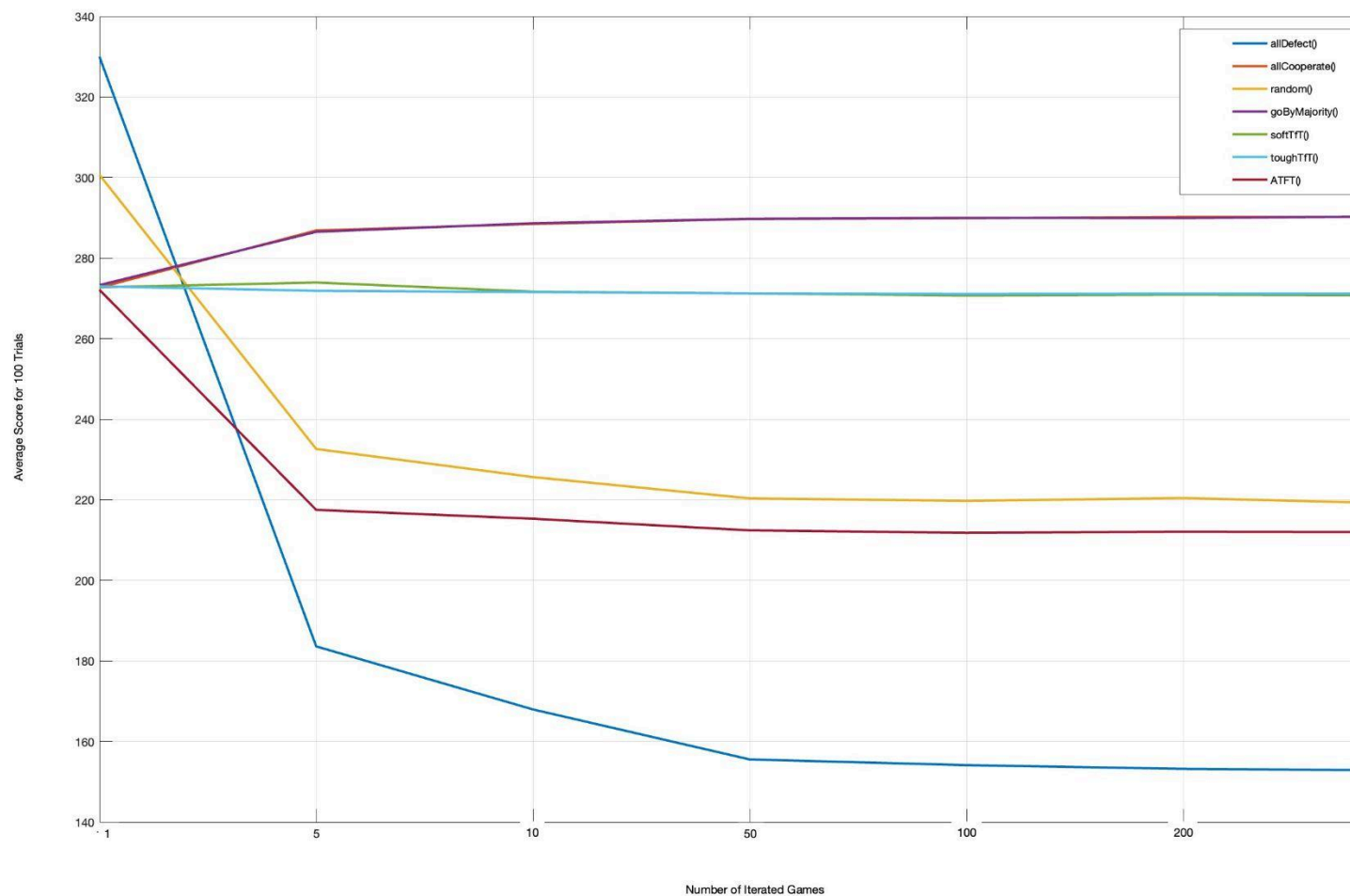
Player 2: 11

Player 3: 11

Average score (100 trials) for each strategy against all combinations of matchups (343 combinations) for N=1, 5, 10, 50, 100, 200 and 500 iterated games.

	allDefect	allCooperate	Random	goByMajority	softTfT	toughTfT	ATFT
Average score (N=1)	330.00	272.68	300.66	272.3	272.76	272.98	272.12
Average score (N=5)	183.61	286.85	232.61	286.51	273.93	271.87	217.50
Average score (N=10)	167.96	288.46	225.64	288.65	271.67	271.55	215.30
Average score (N=50)	155.55	289.71	220.35	289.75	271.23	271.23	212.41
Average score (N=100)	154.13	289.88	219.73	289.98	270.69	271.11	211.8
Average score (N=200)	153.21	290.21	220.21	289.92	270.90	271.18	212.08
Average score (N=500)	152.83	290.23	219.08	290.35	270.72	271.18	211.96





<http://www.prisoners-dilemma.com/common-strategy/>

Results

After examining the outcomes of the games simulated above we can clearly see that some strategies are preferable to others. For example, 'allDefect' is unsurprisingly the least successful strategy out of those tested after 500 trials. This makes sense because of the incentives the other two players have to cooperate; for most other combinations of strategies the other two players will collaborate against the defector causing the defector to lose a majority of the time. On the other hand, strategies that tend to cooperate have a higher success rate after many games. Despite this long term behavior, we can see that for a small number of games the results are quite different. If only a single game is played, strategies that tend to defect have the advantage. This is because multiple of the strategies have a tendency to learn by basing their decisions on the outcomes of previous trials. In the case of the 'allDefect' strategy, depending on the chosen strategy of their competitors, they may learn to distrust the defector over time. Whereas for only one game the fellow players have not yet had this opportunity and are playing against an unknown quantity. This allows the defector to abuse the trust (or ignorance) of their competitors to achieve a favorable outcome in the short term.

An interesting result of this simulation is the fact that the 'softTFT' and 'toughTFT' strategies' success rates converge after many games. This implies that the probability of both competitors defecting and the probability of either competitor defecting also converges over time. This is likely due to the diversity of strategies that the 'softTFT' and 'toughTFT' players are competing against. Other strategies become more or less likely to cooperate after playing against a TFT strategy with their effects canceling out over time. The 'ATFT' strategy is more successful than either of the other TFT strategies due to its ability to adapt to and predict the moves of its competitors. This ability to adapt is most like the behavior of real world players,

easily beating out the competition due to its flexibility compared to the other strategies used in this simulation. Even this strategy is still fairly simple however, with most real life players being able to make more creative judgements and strategies like abusing the trust of other players or feigning cooperation. Despite the simplicity of the strategy compared to real life interactions it is still by far the most successful of those tested in this simulation, showcasing how important the complexity of strategies is in determining the long term success rate.

Many real world behaviors can be explained with the results of this simulation including the abuse of trust of cooperative players, the ability of players to develop distrust of defectors, and the strategic value of complex decision making which is only exaggerated by social dynamics. Most of these lessons can seem quite obvious to most people but this simulation gives an insight into the origin of such behaviors in social competition. As the number of players grows, the kinds of strategies grow exponentially and their success can vary with some strategies being preferable in small groups whereas others may be more successful with larger groups. This would be a fascinating area to explore beyond the scope of this paper's findings.

Conclusion

This model provides an interesting view into the logic of social decision making when the stakes are high for self preservation. For example, as was noted in the results, the ability of the players to begin to suspect and distrust the allDefect strategy after many games parallels the real world logical processes that govern our daily lives. Weighing stakes and gathering observations to make a decision are critical parts of our existence, and without this logic we sometimes take for granted, there would be no way to gain an upper hand in a social competition.

While this model is somewhat limited in complexity, the changing of the effective scores of each strategy with the number of iterations shows that even when given a small amount of information about your competitors, an optimal strategy emerges (in this paper and in the case of many games, the goByMajority strategy). In the real world, however, a player in this game might not have the luxury of knowing 500 iterations of their opponents' strategies, and the resulting "most successful" strategy for going in blind happens to be to defect.

Overall, this model is a great introduction to the premises and computational complexity of game theory, showing that simple games (or seemingly simple social competitions) can have surprising amounts of intricacies and nuance.

Reference

O'Donnell, M. J. (1998). *The Three-Player Prisoner's Dilemma*.

<https://www.classes.cs.uchicago.edu/archive/1998/fall/CS105/Project/node6.html>